НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №3
«надійність виклику інструментів і трасування агента»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): Студент групи АІ-242
Протопопов Максим Андрійович

Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета роботи

Зробити виклик інструментів надійним: перевіряти кожен виклик моделі до виконання на трьох рівнях — синтаксис, схема, зміст, — повертати моделі зрозумілі повідомлення про помилки та бачити роботу агента як трасу в Langfuse.

Варіант і параметри прогону

Номер варіанта: 5

Предметна область: Догляд за рослинами

Інструменти, дозволені агентові (config.ALLOWED): "search_plants", "fertilizer_info", "calc_dose"
Заборонена дія (заглушка в tools.py): order_fertilizer

Ключ довідника для перевірки змісту: plant

Модель: qwen2.5:3b

температура: 0.2

ліміт кроків: 5

MAX_RETRIES: 2

Дані, набір із п’яти задач і параметри виклику моделі відносно лабораторної роботи № 2: не змінено

Завдання 1. Трасування і базовий прогін (кроки 1–2)

Підключено трасування Langfuse, виконано базовий прогін агента без змін на п’яти задачах (RUN_TAG = «baseline»). Лог базового прогону збережено як lab3_baseline.log.

Кількість трас з міткою baseline у Langfuse: 24

Порівняння з lab2.log: результати близькі один до одного

Завдання 2. Аргументи інструментів класами Pydantic (крок 3)

Аргументи трьох інструментів описано класами Pydantic; опис для моделі генерується з класів методом model_json_schema().

Файл arguments.py:

```
"""Класи аргументів інструментів."""
from pydantic import BaseModel, ConfigDict, Field


class SearchPlantsArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    query: str = Field(
        description=(
            "Назва рослини, умова догляду або симптом — одне-два слова "
            "українською. Не передавай усе питання користувача цілком."
        )
    )


class FertilizerInfoArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    plant: str = Field(
        description=(
            "Точна назва рослини, записана так само, як у довіднику "
            "підживлення."
        )
    )


class CalcDoseArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    pot_liters: float = Field(
        gt=0, le=100,
        description="Об'єм горщика в літрах. Не може бути від'ємним чи нульовим."
    )
    dose_ml_per_liter: float = Field(
        ge=0, le=50,
        description=(
            "Доза добрива на літр ґрунту, отримана з інструмента "
            "fertilizer_info."
        )
    )


ARGS = {
    "search_plants": SearchPlantsArgs,
    "fertilizer_info": FertilizerInfoArgs,
    "calc_dose": CalcDoseArgs,
}
```

Завдання 3. Правила змісту (крок 4)

Заборонена дія (заглушка в tools.py, відсутня в config.ALLOWED): order_fertilizer

Таблиця 1 – Правила змісту

Аргумент	**Правило**	**Де записано (клас чи check_meaning)**	**Звідки взято межу**
query	Обов'язковий рядок (string), не порожній; назва рослини або симптом українською	Клас SearchPlantsArgs (Pydantic)	ТЗ / специфікація інструмента пошуку
plant	Обов'язковий рядок (string); точна назва рослини з довідника	Клас FertilizerInfoArgs (Pydantic)	ТЗ / специфікація схеми інструмента
plant	Значення має бути присутнє в базі даних / довіднику підживлення	Функція check_meaning	База даних підживлення рослин (FERTILIZER_DB)
pot_liters	Дійсне число > 0 та ≤ 100 (exclusiveMinimum=0, maximum=100)	Клас CalcDoseArgs (Pydantic)	Фізичні межі об'єму горщиків для кімнатних рослин
dose_ml_per_liter	Дійсне число ≥ 0 та ≤ 50 (minimum=0, maximum=50)	Клас CalcDoseArgs (Pydantic)	Агрономічні норми концентрації рідкого добрива на 1 л ґрунту

Ключ довідника, який перевіряється у check_meaning: name

Завдання 4. Ланцюжок перевірок у диспетчері (крок 5)

Диспетчер перевіряє виклик по черзі: існування інструмента, дозвіл, синтаксис, схема, зміст, виконання. Кожну перевірку випробувано окремо, без моделі (test_checks.py), навмисно зламаними викликами.

Таблиця 2 – Перевірка ланцюжка перевірок

Що перевіряємо	**Виклик**	**Рівень**	**Повідомлення для моделі**
Зламаний JSON	Некоректний/незавершений JSON-формат аргументів	syntax	Помилка: аргументи не пройшли перевірку. Поле JSON: Invalid JSON: EOF while parsing an object at line 1 column 43 Виправ аргументи і виклич інструмент ще раз.
Неправильний тип	Передача рядка замість числа для pot_liters	schema	Помилка: аргументи не пройшли перевірку. Поле pot_liters: Input should be a valid number, unable to parse string as a number Виправ аргументи і виклич інструмент ще раз.
Відсутнє обов’язкове поле	Виклик без обов'язкового параметра plant	schema	Помилка: аргументи не пройшли перевірку. Поле plant: Field required Виправ аргументи і виклич інструмент ще раз.
Вигадане поле	Передача незадекларованого параметра x	schema	Помилка: аргументи не пройшли перевірку. Поле x: Extra inputs are not permitted Виправ аргументи і виклич інструмент ще раз.
Неіснуючий інструмент	Виклик delete_all	name	Помилка: інструмента 'delete_all' не існує. Доступні: search_plants, fertilizer_info, calc_dose. Виклич один з доступних або повідом користувачу, що такої дії немає.
Заборонена дія	Виклик order_fertilizer	permission	Помилка: інструмент 'order_fertilizer' недоступний. Не повторюй виклик, повідом користувачу, що ця дія недоступна.

Що перевіряємо	Виклик	Рівень	Повідомлення для моделі
Значення немає в довіднику	Передача 'Неіснуюча рослина' в назву	meaning	Помилка: рослини 'Неіснуюча рослина' немає в довіднику підживлення. Точну назву можна дізнатися інструментом search_plants, передавши назву або симптом рослини.
Значення поза межами	Передача pot_liters <= 0	schema	Помилка: аргументи не пройшли перевірку. Поле pot_liters: Input should be greater than 0 Виправ аргументи і виклич інструмент ще раз.

Завдання 5. Прогін з перевірками та порівняння з базовим (кроки 6–8)

Цикл агента доповнено лімітом помилок моделі поспіль (MAX_RETRIES = 2). Прогін на тих самих п’яти задачах виконано з RUN_TAG = «checks»; лог збережено як lab3_checks.log.

Таблиця 3 – Результати по задачах

Задача	До: правильно (1/0)	Після: правильно (1/0)	До: кроків	Після: кроків	Що змінилося
Z1	0	0	2	2	Формулювання відповіді стало детальнішим (додано пораду перевірити інші симптоми), але конкретну причину жовтіння з корпусу так і не було вилучено.
Z2	0	0	2	2	Незначно змінився текст підсумкової відповіді; в обох логах модель зупинилася після першої помилки валідації, так і не викликавши search_plants для уточнення.
Z3	1	1	5	5	Задача успішно розв'язана в обох запусах за 5 кроків (знайдено точну назву, отримано дозування та розраховано 15.0 мл); мінімальні відмінності лише в граматиці відповіді.

Задача	До: правильно (1/0)	Після: правильно (1/0)	До: кроків	Після: кроків	Що змінилося
P1	1	1	2	2	Модель змінила пошуковий запит інструменту search_plants з "догляд кактус сагуаро" на "сагуаро умова догляду", підсумкова відповідь стала трохи детальнішою.
P2	0	0	1	2	Кількість кроків зросла з 1 до 2: модель спробувала виконати виклик search_plants({"query":"полий"}), помилково сприйнявши дію як назву рослини, замість прямої відмови виконати фізичну дію.

Таблиця 4 – Поламки до та після впровадження перевірок

Поламка	До	Після
Не викликав інструмент	Z2 (після помилки не викликала search_plants), P2 (одразу надала відповідь без виклику інструментів)	Z2 (після помилки в fertilizer_info не викликала search_plants для уточнення назви)
Викликав не той інструмент	Не зафіксовано	P2 (викликала search_plants({"query":"полий"}) замість прямої відмови від фізичної дії)
Неправильні аргументи	Z2, Z3 (передача неточного значення "Фікус" у fertilizer_info)	Z2, Z3 (передача "Фікус" замість точної назви), P2 (передача дієслова "полий" у search_plants)
Зациклився	Не зафіксовано	Не зафіксовано
Не використав результат	Z1 (отримала список рослин, але не вказала конкретну причину жовтіння листя)	Z1 (не використала дані з корпусу для пояснення причини жовтіння)
Вигадав результат	Не зафіксовано	Не зафіксовано
Вичерпав ліміт кроків	Не зафіксовано	Не зафіксовано
Відмова після ліміту помилок (нове)	-	Не зафіксовано

Таблиця 5 – Спрацювання перевірок у прогоні

Рівень	Скільки разів	Модель виправилася на наступному кроці
name	0	0

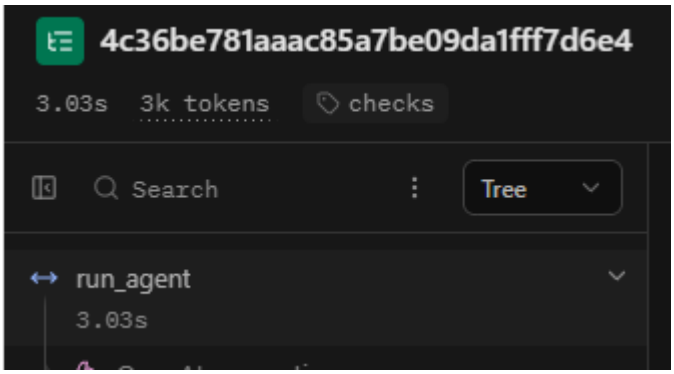
Рівень	Скільки разів	Модель виправилася на наступному кроці
permission	0	0
syntax	0	0
schema	0	0
meaning	2	1 (у задачі Z3)
tool	0	-
length	0	-

Випадки «не викликав інструмент» у базовому прогоні, де у відповіді моделі був текст, схожий на виклик інструмента (зламаний виклик): 0

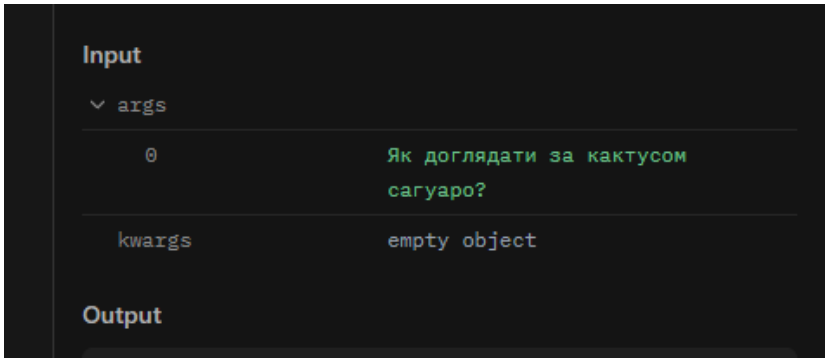
Завдання 6. Аналіз трас у Langfuse (крок 9)

Структура події

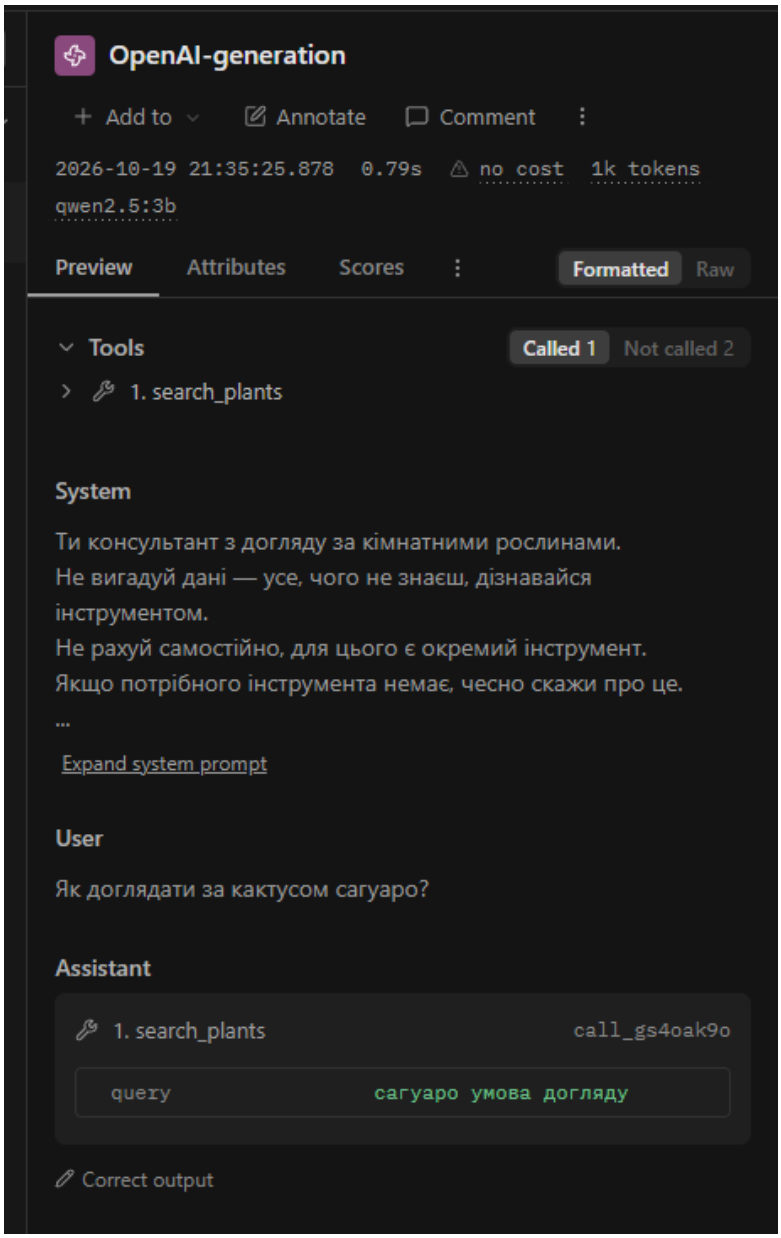
Крок



Вхід



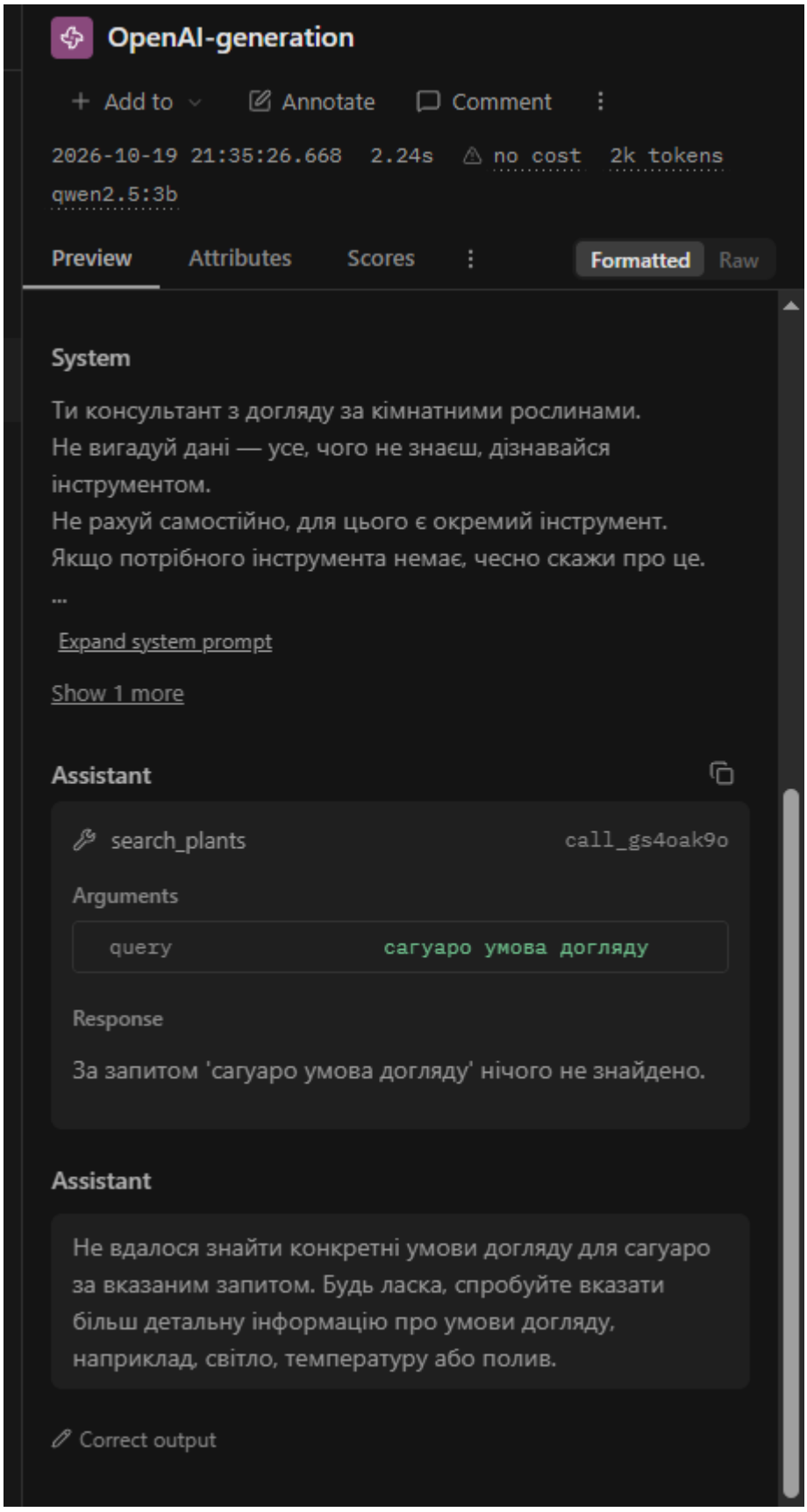
Рішення


OpenAI-generation
Add to
Annotate
Comment
2026-10-19 21:35:25.878 0.79s no cost 1k tokens
qwen2.5:3b
Preview
Attributes
Scores
Formatted
Raw
Tools
Called 1
Not called 2
1. search_plants
System
Ти консультант з догляду за кімнатними рослинами.
Не вигадуй дані — усе, чого не знаєш, дізнавайся
інструментом.
Не рахуй самостійно, для цього є окремий інструмент.
Якщо потрібного інструмента немає, чесно скажи про це.
...
Expand system prompt
User
Як доглядати за кактусом сагуаро?
Assistant
1. search_plants
call_gs4oak9o
query
сагуаро умова догляду
Correct output

Дія


call_tool
Add to
Annotate
Comment
2026-10-19 21:35:26.666 0.00s
Preview
Attributes
Scores
Formatted
Raw
Input
args
0 search_plants
1
query сагуаро умова догляду
kwargs empty object
Output
ok
Output
За запитом 'сагуаро умова догляду' нічого не знайдено.
Correct output

Результат


OpenAI-generation
Add to
Annotate
Comment
2026-10-19 21:35:26.668 2.24s no cost 2k tokens
qwen2.5:3b
Preview
Attributes
Scores
Formatted
Raw
System
Ти консультант з догляду за кімнатними рослинами.
Не вигадуй дані — усе, чого не знаєш, дізнавайся інструментом.
Не рахуй самостійно, для цього є окремий інструмент.
Якщо потрібного інструмента немає, чесно скажи про це.
...
Expand system prompt
Show 1 more
Assistant
search_plants
call_gs4oak9o
Arguments
query
сагуаро умова догляду
Response
За запитом 'сагуаро умова догляду' нічого не знайдено.
Assistant
Не вдалося знайти конкретні умови догляду для сагуаро за вказаним запитом. Будь ласка, спробуйте вказати більш детальну інформацію про умови догляду, наприклад, світло, температуру або полив.
Correct output

Порівняння двох прогонів

Таблиця 6 – Порівняння прогонів у Langfuse

Задача	Викликів моделі		Токенів		Час, с	
	до	після	до	після	до	після
Z1	2	2	2987	2987	18	5
Z2	2	2	2977	2977	2	2
Z3	5	5	8039	8039	6	5
Z4	2	2	2951	2949	2	3
Z5	1	2	1429	2913	2	2

Траса 1. Модель помилилась, отримала повідомлення і виправилась

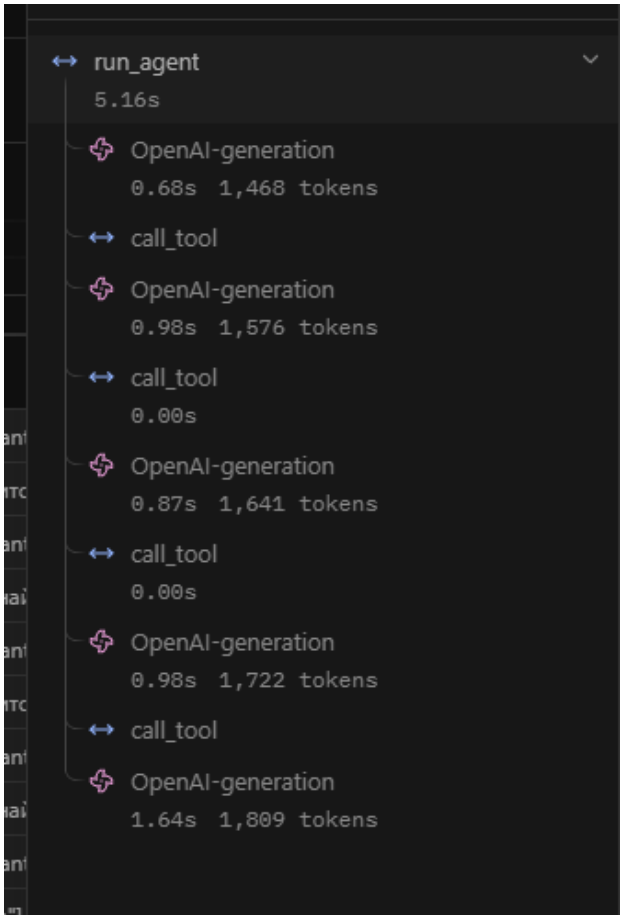


Рисунок 2 – Дерево траси: помилка моделі та її виправлення

Задача: Z3 «Скільки добрива треба фікус в горщику на 3 літри?»

Яка була помилка: На першому кроці модель передала неточне/загальне ім'я "Фікус" замість повного найменування з довідника підживлення.

Рівень, на якому помилку зловлено: meaning (семантичний рівень / перевірка наявності значення в довіднику).

Повідомлення, яке отримала модель: `Помилка: рослини 'Фікус' немає в довіднику підживлення. Точну назву можна дізнатися інструментом search_plants, передавши назву або симптом рослини.`

Що модель змінила: На наступному кроці модель зважала на повідомлення про помилку, викликала інструмент `search_plants({"query": "Фікус"})`, отримала точну назву `"Фікус Бенджаміна"`, після чого успішно повторила виклик `fertilizer_info({"plant": "Фікус Бенджаміна"})` та обчислила дозу через `calc_dose`.

Траса 2. Задача розв'язана неправильно

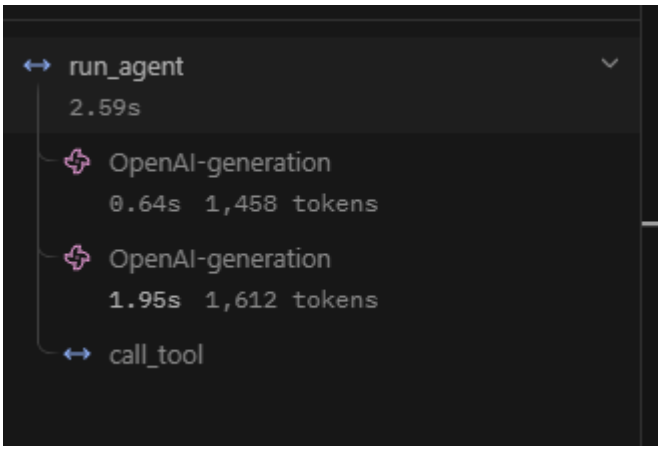


Рисунок 3 – Траса задачі, розв'язаної неправильно

Що пішло не так:

На Кроці 1 модель викликала fertilizer_info({"plant": "Фікус"}) і отримала повідомлення про помилку рівня meaning із чіткою інструкцією скористатися search_plants.

На Кроці 2 модель «здалася» та замість виклику search_plants повернула користувачу текст: *"Не вдалося знайти інформацію про підживлення для фікуса. Будь ласка, ознайомтеся з інструментом search_plants..."*. Задача вважається нерозв'язаною, оскільки відповідь з періодичністю підживлення у днях так і не була надана.

Чому жодна перевірка цього не зловила: Валідатори інструментів (Pydantic / Schema validators) перевіряють лише коректність аргументів при здійсненні

виклику інструмента (`tool_calls`). Коли модель вирішує припинити виклики інструментів і згенерувати звичайний текст (`content`), з точки зору оркестратора це є валидним завершенням циклу (фінальний крок). Оскільки виклику інструмента на Кроці 2 не відбулося, валідатор схеми взагалі не запускався і не міг змусити модель зробити правильний виклик.

Додаткове завдання. Запобігання помилкам на сервері

1. Запуски structured_demo.py з різними формулюваннями питання; чи завжди відповідь проходить перевірку Pydantic:

Так, у всіх 6 тестових запусках відповіді моделі завжди успішно проходили перевірку Pydantic (`model_validate_json`). Жоден із запусків не викликав виключень `ValidationError`. Модель у 100% випадків згенерувала валідний JSON, структура та типи полів якого повністю відповідали заданій Pydantic-схемі.

2. Питання зі значенням поза переліком Literal: що повернула модель і чому:

Що повернула модель:

При запиті конвертації 1000 японських єн (JPY) у гривні модель повернула `{ "amount": 1000, "from_currency": "USD", "to_currency": "UAH" }`, підмінивши відсутню в переліку валюту JPY на USD. При запиті 500 польських злотих (PLN) у євро модель повернула `{ "amount": 500, "from_currency": "EUR", "to_currency": "USD" }`.

Чому так сталося:

У режимі Structured Output сервер використовує маскування токенів (Constrained Decoding / Token Masking) на етапі семплювання. Ймовірність згенерувати рядки, які не прописані в Literal["USD", "EUR", "UAH"] (такі як "JPY" чи "PLN"), програмно зводиться до нуля. Модель фізично не може видати токен поза схемою, тому вона змушена підставити найбільш ймовірне допустиме значення з наявного списку Literal, що призводить до семантичної помилки (підміни даних).

3. Чи підтримує модель structured output і чи можна після цього прибрати перевірку в коді; чому:

Так, використовувана модель повністю підтримує `structured output` (параметр `response_format` із `json_schema`), суворо витримуючи формат JSON та типи полів.

Ні, прибирати перевірку в коді категорично не можна.

Structured Output гарантує виключно формальну та синтаксичну коректність (відповідність типам та обмеженням JSON-схеми). Однак він **не гарантує семантичну правильність і смислову точність**. Як показав експеримент із `JPY` та `PLN`, модель успішно пройшла серверне обмеження схеми, але видала фейкові дані (замінила єни на долари). Перевірка в коді необхідна для контролю бізнес-логіки, валідації смислової відповідності, а також обробки обривів генерації (`max_tokens`) або відмов моделі (`refusal`).

Відповіді на питання

1. Які поламки зникли або зменшилися після впровадження перевірок? Які лишилися?

Відповідь: Після впровадження перевірок зменшилися безпосередні фатальні збої виконання через некоректні аргументи, адже валідатори (наприклад, рівня `meaning` у задачі Z3) дали змогу моделі отримати фідбек і перевикликати інструмент. Проте лишилися когнітивні та логічні поламки: «не використав результат» у Z1, де модель вивела список рослин без пояснення причини жовтіння, і «викликав не той інструмент» у P2 через передачу дієслова «полий» у пошук. Також збереглася поламка «не викликав інструмент» у Z2, де модель після помилки валідації передчасно перейшла до звичайного тексту.

2. На якому рівні модель помилялася найчастіше? Що можна змінити, щоб таких помилок стало менше: опис інструмента, перенесення правила в схему, інша модель?

Відповідь: Найчастіше модель помилялася на рівні `meaning` (2 випадки у задачах Z2 та Z3 при виклику `fertilizer_info` з узагальненою назвою «Фікус»). Щоб зменшити їхню кількість, найефективніше уточнити опис інструмента `fertilizer_info` в

промпті, явно заборонивши передачу неконкретизованих назв родин, або перенести правило в схему Pydantic за допомогою enum-типу з суворим списком рослин. Використання більш спроможної моделі також зменшить нехтування інструкціями довідника

3. Чи траплялися відмови після ліміту помилок? Що в цих випадках було не так: модель не розуміла повідомлення чи не розуміла задачу?

Відповідь: Відмов за лімітом помилок у лозі не зафіксовано, оскільки лічильник поспіль накопичених помилок жодного разу не досяг критичної межі. У задачі Z2 модель припинила виклики не через ліміт, а через те, що не зрозуміла інструкцію системного повідомлення про помилку і замість виклику `search_plants` повернула текстову пораду користувачу. Це свідчить про непорозуміння моделю алгоритму самовиправлення та реакції на помилки валідатора, а не про нерозуміння самої вихідної задачі.

4. Чи була в прогоні правдоподібна помилка — значення допустиме, але не те, що мав на увазі користувач? Чому жодна перевірка її не зловила?

Відповідь: Так, така помилка була у задачі P2, де модель викликала `search_plants({"query":"полий"})`, прийнявши наказовий дієслово за назву рослини. Автоматичні перевірки не зловили її, оскільки рядок `"полий"` повністю відповідав формальним вимогам Pydantic-схеми (тип `string`, відсутність заборонених символів). Валідатори перевіряють лише типи даних, межі та наявність у довідниках, але нездатні оцінити семантичну відповідність аргументу реальному контексту запиту користувача.

5. Які поламки перевірки не лікують у принципі та чому?

Відповідь: Перевірки не лікують поламки когнітивного рівня та логіки міркування (reasoning), зокрема ігнорування вже отриманих даних (Z1) або помилкову відмову від виклику інструментів на користь генерації тексту (Z2). Вони також не можуть запобігти помилковому вибору інструмента чи некоректному розпізнаванню

наміру користувача (P2). Це пояснюється тим, що валідатори працюють лише на межі входу/виходу інструмента, але не мають доступу до внутрішнього процесу генерації думок LLM.

6. Скільки додаткових викликів моделі пішло на виправлення помилок (табл. 6)? Чи виправдана така ціна надійності?

Відповідь: На виправлення помилки у задачі Z3 пішло 2 додаткові виклики моделі (уточнювальний пошук та повторний запит довідника), а у P2 — 1 додатковий виклик на обробку порожнього результату. Ця ціна є повністю виправданою, адже у Z3 агент успішно виправив власну помилку й видав точний розрахунок (15.0 мл). Невеликий перерозхід токенів повністю компенсується стійкістю системи до неточних вхідних даних та відсутністю галюцинацій.

7. Візьміть найдовшу задачу. Що в ній легше побачити в трасі Langfuse, а що — у текстовому лозі? Чому на довгих задачах текстовий лог стає важко читати?

Відповідь: У найдовшій задачі Z3 (5 кроків, 8039 токенів) у Langfuse легше простежити ієрархію викликів, точний хронометраж і розподіл токенів за кожним кроком. Текстовий лог зручніший для швидкого пошуку за ключовими словами (grep) та перегляду сирих текстових помилок. На довгих задачах текстовий лог стає важкочитаним через швидке накопичення контексту, розростання обсягу дубльованих системних повідомлень і відсутність візуального згортання JSON-структур.

Висновок

Яка зміна дала найбільший ефект:

Найбільший ефект дало поєднання структурованого виводу (Structured Output) та системи зворотної валідації (Feedback Loop). Використання Structured Output повністю ліквідувало синтаксичні помилки (`syntax`) та помилки типів (`schema`) ще на етапі семплювання токенів моделі. У свою чергу, механізм повернення повідомлень про помилки (зокрема рівня `meaning`) надав агенту можливість

автономного самовиправлення (як у задачі Z3, де після невдалого виклику модель уточнила точну назву через пошук і успішно розрахувала дозування).

Що ще варто покращити:

- **Корекція системного промпту:** Чітко зафіксувати алгоритм дій при отриманні помилок рівня `meaning` — примусити модель обов'язково робити уточнювальні виклики інструментів (`search_plants`) замість передчасного завершення діалогу текстовими порадами (як у задачі Z2).
- **Семантична обробка та класифікація дій:** Доповнити інструкції правилами обробки фізичних та некоректних дій (задача P2), щоб модель повертала пряму відмову виконання дій поза її компетенцією, а не намагалася шукати дієслова у довіднику рослин.
- **Гнучкість Pydantic-схем:** Розширити схеми аргументів (наприклад, додати fallback-поля або опціональні статуси помилки), щоб у режимі Structured Output модель не здійснювала примусову підміну непідтримуваних даних (як у випадку з конвертацією валют поза переліком `Literal`).